

Desarrollo de aplicaciones interactivas

R3

Curso 2026/2027

Jesús Zamora Jiménez

Contenido

1. Concepto del proyecto 3
1.1. Descripción 3
1.2. Objetivos 3
1.3. Idea visual 3
2. Casos de uso 3
3. Bocetos de interfaz 4
3.1. Estructura de la ventana emergente 4
3.2. Evolución de la animación 4
3.3. Resultado implementado 5
4. Descripción de las funcionalidades 5
4.1. Temporizador y estados 5
4.2. Visualización generativa 6
4.3. Interfaz 6
5. Estructura de archivos y manifiesto 6
5.1. Archivos del proyecto 6
5.2. Manifiesto 7
6. Almacenamiento persistente y recuperación del estado 7
6.1. El problema 7
6.2. La solución: marcas temporales 7
6.3. Datos guardados 8
6.4. Permisos 8
7. Instalación y pruebas en Firefox 8
7.1. Cargar la extensión 8
7.2. Plan de pruebas 9
8. Propuesta de mejora futura: notificaciones 9
8.1. Utilidad 9
8.2. Permisos necesarios 9
8.3. Documentación técnica 9
9. Reflexión sobre el proceso de desarrollo 10
10. Enlace a Github 10
11. Enlace a Folio 10
12. Fuentes 10

1. Concepto del proyecto

1.1. Descripción

Focus Flow es una extensión para Firefox que ayuda a organizar el trabajo en periodos de concentración seguidos de descansos breves, en la línea de la técnica Pomodoro. Se abre desde el icono de la barra de herramientas y muestra, en una ventana emergente pequeña, el estado actual, el tiempo restante y una animación generativa hecha con p5.js que representa el paso del tiempo.

Aunque el nombre no es muy original (no he querido reinventar la ruda) si he planteado la extensión de forma sencilla y funcional donde hay pocas opciones; son controles convencionales y un código fácil de leer. La prioridad es que el temporizador sea fiable y no pierda la cuenta, aunque se cierre la ventana.

1.2. Objetivos

- Gestionar sesiones de concentración de 25, 45 o 60 minutos y descansos de 5 minutos.
- Poder iniciar, pausar, reanudar y reiniciar el temporizador.
- Representar el paso del tiempo con una animación generativa en p5.js.
- Conservar el estado entre aperturas con storeItem() y getItem().
- Funcionar como extensión Manifest V3 de Firefox, sin cargar código externo.

1.3. Idea visual

La animación parte de la idea de un reloj creativo abstracto, donde en lugar de agujas y números el tiempo se expresa con un anillo que se va completando, un círculo central que crece, un punto que gira una vuelta por minuto y diez círculos que se rellenan de diez en diez por ciento. Es una segunda lectura del tiempo que permite ver de un vistazo cuánto se ha avanzado.

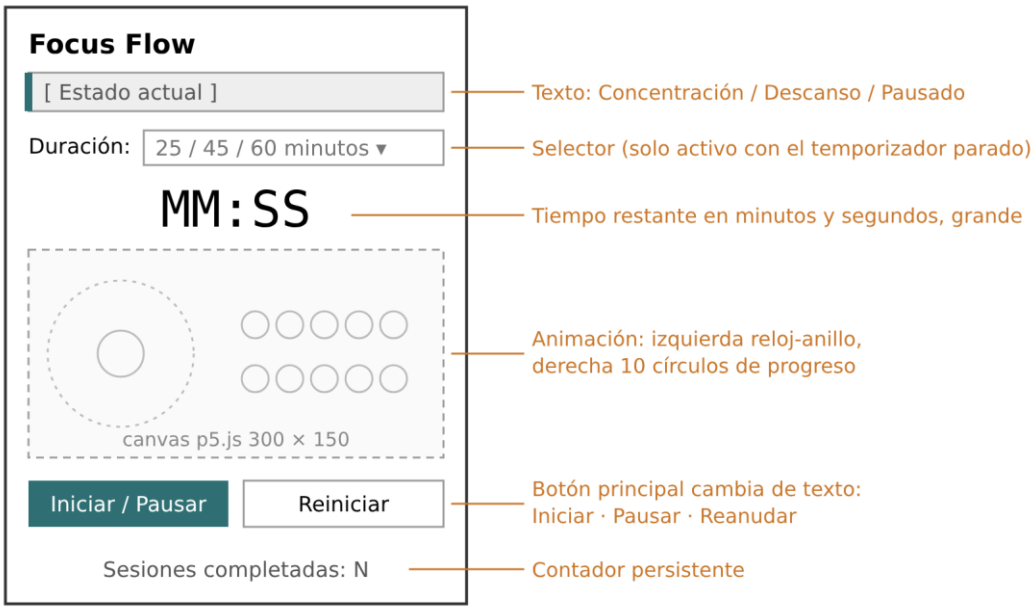
2. Casos de uso

Caso de uso	Descripción y resultado esperado
CU1. Iniciar una sesión	Elijo 25, 45 o 60 minutos y pulso Iniciar. El estado pasa a “Concentración”, el tiempo empieza a descontar y la animación comienza a evolucionar.
CU2. Pausar y reanudar	Al pulsar Pausar el tiempo se congela, el estado indica “Pausado” y la animación se queda quieta en gris. Reanudar continúa desde el mismo punto.
CU3. Reiniciar	Vuelve a una sesión de concentración completa y parada. Si estaba en descanso, el descanso termina. El contador de sesiones se conserva.
CU4. Cerrar y volver a abrir	Inicio una sesión, cierro la ventana y la abro más tarde. El temporizador muestra el tiempo correcto, descontando también el tiempo que estuvo cerrada.
CU5. Fin de la concentración	Al terminar la sesión (con la ventana abierta o cerrada) se suma una sesión completada y se prepara el descanso “Descanso (pulsa Iniciar)”.
CU6. Descansar y volver	Pulso Iniciar para empezar el descanso de 5 minutos. Al terminar, la extensión vuelve a “Concentración (pulsa Iniciar)”.
CU7. Cambiar la duración	Con el temporizador parado elijo otra duración; el tiempo se actualiza y se recuerda. Mientras corre o está en pausa, el selector está desactivado.

3. Bocetos de interfaz

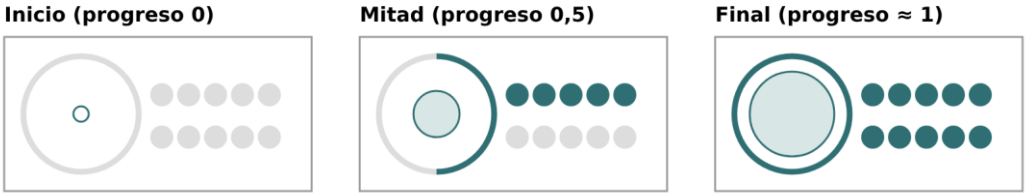
3.1. Estructura de la ventana emergente

Antes de programar pensé una distribución vertical simple, ordenada de arriba abajo según lo que el usuario lee y hace. He querido representar qué está pasando (estado), cuánto falta (tiempo y animación) y qué puede hacer (botones).



Boceto 1. Estructura de la ventana emergente.

3.2. Evolución de la animación



· Anillo: se va completando con color según el porcentaje del periodo que ha pasado.
· Disco central: crece con el progreso. Punto exterior: da una vuelta por cada minuto transcurrido.
· Círculos de la derecha: cada uno representa un 10 % del periodo y se rellena al alcanzarlo.
· Color: verde azulado = concentración, naranja = descanso, gris = pausa.

Boceto 2. Evolución prevista de la animación a lo largo de un periodo.

3.3. Resultado implementado



Capturas de la extensión: concentración, pausa y descanso.

4. Descripción de las funcionalidades

4.1. Temporizador y estados

El temporizador se controla con dos variables:

- Tipo de periodo (tipo): concentracion o descanso.
- Situación (estado): parado, en marcha o pausado.

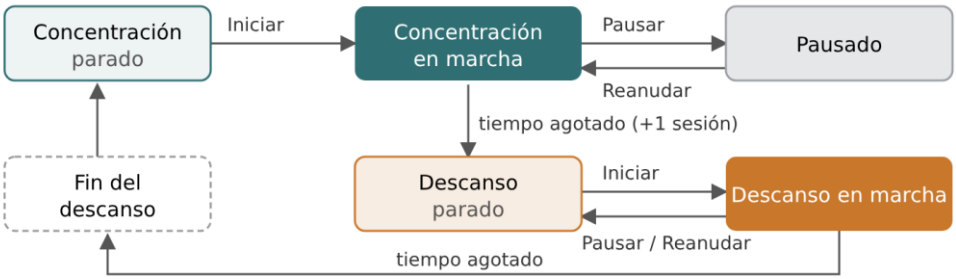


Diagrama de estados. «Reiniciar», desde cualquier estado, lleva a «Concentración parado».

Cuando un periodo termina el siguiente queda preparado pero no arranca solo, senecesita que el usuario inicie el descanso y cuándo empezar la siguiente sesión. Así, si la ventana estuvo cerrada mucho tiempo, no se encadenan periodos que nadie ha visto.

El indicador de estado muestra “Concentración”, “Descanso” o “Pausado”, y añade “(pulsa Iniciar)” cuando el temporizador está parado. El botón principal cambia entre Iniciar, Pausar y Reanudar, y la animación usa un color distinto en cada caso; verde azulado, naranja y gris.

4.2. Visualización generativa

El archivo sketch.js usa p5.js en modo global con un canvas de 300 por 150 píxeles a 30 fotogramas por segundo. La función dibujarReloj() dibuja:

- Un anillo que se completa con arc() según el progreso.
- Un círculo central que crece.
- Un punto que da una vuelta por cada minuto transcurrido.
- Diez círculos a la derecha que se rellenan de 10 en 10 %.

Todo se calcula a partir del progreso del periodo (de 0 a 1) y del tiempo transcurrido, de esta manera, en pausa la imagen se queda congelada y al cerrar y volver a abrir, se dibuja exactamente la misma composición que corresponde a ese momento.

4.3. Interfaz

El esqueleto de la ventana está en popup.html que contiene el título, estado, tiempo, contenedor del canvas y contador de sesiones. El selector de duración y los dos botones se crean mediante el DOM con las funciones de p5 createSelect() y createButton(), se colocan en su sitio con parent() y reciben sus estilos desde style.css.

No hay temporizadores de JavaScript aparte ya que la función draw() de p5 llama en cada fotograma a comprobarFinal(), actualizarPantalla() y dibujarReloj().

5. Estructura de archivos y manifiesto

5.1. Archivos del proyecto

Archivo	Función
manifest.json	Es la configuración de la extensión. Contiene ventana emergente, permisos y ajustes de Firefox.
popup.html	Estructura de la ventana emergente y carga de los scripts.
style.css	Estilos de la interfaz.
popup.js	Datos del temporizador, cálculos de tiempo, guardado, botones y textos.
sketch.js	setup() y draw() de p5 y la animación del reloj.
p5.min.js	Biblioteca p5.js.
icon.svg	icono del botón en la barra de herramientas.

Las funciones de p5 (storeItem, getItem, createButton…) solo están disponibles cuando p5 se ha inicializado. Por eso popup.js solo define variables y funciones, y es setup(), en sketch.js, quien llama a cargarDatos() y crearControles().

5.2. Manifiesto

```
{
 "manifest_version": 3,
 "name": "Focus Flow",
 "version": "1.0.0",
 "description": "Temporizador de concentración y descanso con una animación generativa en p5.js.",
 "icons": { "48": "icon.svg", "96": "icon.svg" },
 "action": {
  "default_title": "Focus Flow",
  "default_popup": "popup.html",
  "default_icon": "icon.svg"
 },
 "permissions": ["storage"],
 "browser_specific_settings": {
  "gecko": {
   "id": "focus-flow@example.org",
   "strict_min_version": "140.0",
   "data_collection_permissions": { "required": ["none"] }
  }
 }
}
```

- action.default_popup: en Manifest la ventana emergente se declara en action.
- permissions: ["storage"]: permiso solicitado en el proyecto (ver apartado 6.4).
- gecko.id: identificador de la extensión.
- data_collection_permissions: declara que la extensión no recoge datos personales.
- strict_min_version: "140.0": primera versión de Firefox de escritorio que entiende la clave anterior.
- CSP: se aplica la política por defecto de Manifest ya que solo permite scripts de la propia extensión.

6. Almacenamiento persistente y recuperación del estado

6.1. El problema

Firefox destruye la página de la ventana emergente cada vez que se cierra, por lo tanto se pierden las variables y se detiene el código.

6.2. La solución: marcas temporales

En lugar de contar segundos, guardo cuánto tiempo quedaba (restante) y en qué momento se puso en marcha (inicio). El tiempo que queda se calcula siempre con el reloj del sistema:

- tiempo que queda = restante - (Date.now() - inicio) // si está en marcha
- tiempo que queda = restante // si está parado o pausado
- Iniciar o reanudar: inicio = Date.now(); restante no cambia.
- Pausar: se calcula lo que queda, se guarda en restante y el estado pasa a pausado. Mientras está pausado no se usa inicio, así que el tiempo no se descuenta aunque la ventana esté cerrada horas.
- Abrir la ventana: se leen los datos, se recalcula el tiempo y, si el periodo ya terminó, se pasa al siguiente.
- Con la ventana abierta: draw() refresca la pantalla y comprueba si el periodo ha terminado.

6.3. Datos guardados

Uso storeItem(clave, valor) y getItem(clave) de p5.js, que escriben en localStorage. Si un dato todavía no existe, getItem() devuelve null y la función leer() usa un valor por defecto. La función guardarDatos() solo se llama cuando algo cambia, no en cada fotograma.

Clave	Contenido	Ejemplo
duración	Duración de concentración elegida (minutos)	25
estado	Estado del temporizador	en marcha
tipo	Tipo de periodo actual	concentracion
inicio	Momento del último inicio o reanudación (ms desde 1970)	1793900000000
restante	Tiempo que quedaba en ese momento (ms)	1500000
sesiones	Sesiones de concentración completadas	3

6.4. Permisos

El permiso storage está declarado en el manifiesto, tal como pide el enunciado.

Si quiero remarcar que, para las pruebas, al cargar la extensión de forma temporal desde about:debugging, Firefox le asigna un identificador interno nuevo cada vez. Los datos se conservan entre aperturas de la ventana, pero empiezan de cero si se quita y se vuelve a cargar la extensión o se reinicia Firefox.

7. Instalación y pruebas en Firefox

7.1. Cargar la extensión

1. Descomprimir Focus_Flow_extension.zip en una carpeta.
2. Abrir Firefox y escribir about:debugging#/runtime/this-firefox en la barra de direcciones.
3. Pulsar Cargar complemento temporal… y seleccionar el archivo manifest.json.
4. Abrir el menú de extensiones (icono de pieza de puzle) y fijar Focus Flow en la barra de herramientas.
5. Hacer clic en el icono para abrir la ventana emergente.

Para depurar, en about:debugging se pulsa Inspeccionar: la consola muestra los errores y en Almacenamiento --> Almacenamiento local se ven los datos guardados.

7.2. Plan de pruebas

Prueba	Resultado esperado
1. Abrir por primera vez	"Concentración (pulsa Iniciar)", 25:00, contador a 0.
2. Iniciar, cerrar y reabrir al cabo de unos minutos	El tiempo ha bajado lo que duró cerrada.
3. Pausar, cerrar, esperar y reabrir	Mismo tiempo que al pausar; "Pausado". La animación queda gris y quieta.
4. Reanudar	Continúa desde el mismo punto.
5. Dejar terminar una sesión	Suma una sesión y muestra "Descanso (pulsa Iniciar)" con 05:00.
6. Iniciar el descanso y dejarlo terminar	Vuelve a "Concentración (pulsa Iniciar)".
7. Cambiar a 45 minutos con el temporizador parado, cerrar y reabrir	Muestra 45:00 y el selector conserva 45.
8. Intentar cambiar la duración con el temporizador en marcha	El selector está desactivado.
9. Reiniciar durante una sesión o un descanso	Concentración completa y parada. El contador se mantiene.
10. Revisar la consola del inspector	Sin errores ni avisos de seguridad.

8. Propuesta de mejora futura: notificaciones

8.1. Utilidad

Ahora mismo solo se sabe que una sesión ha terminado si se abre la ventana. Una notificación tipo pop up al terminar la concentración ("Sesión completada: toca descansar") o el descanso ("Descanso terminado") permitiría trabajar sin vigilar el temporizador, que es justo el objetivo de la herramienta.

8.2. Permisos necesarios

1. notifications: mostrar notificaciones del sistema.
2. alarms: programar el aviso aunque la ventana esté cerrada.
3. storage: serviría para compartir el estado con el script en segundo plano.

8.3. Documentación técnica

- MDN Web Docs: API notifications y API alarms.
- MDN Web Docs: clave background del manifest.json y guía de scripts en segundo plano.
- MDN Web Docs: API storage y clave permissions.
- Firefox Extension Workshop: guía de migración a Manifest V3.

9. Reflexión sobre el proceso de desarrollo

La decisión que por así decirlo orientó la manera de enfocar la práctica fue tratar el tiempo como un cálculo y no como un contador. Entender que la ventana emergente se destruye al cerrarse, guardar "cuánto quedaba y cuándo empezó" resolvió a la vez la persistencia, la pausa y la recuperación, y simplificó el resto del código.

La misma idea guio la animación ya que el dibujo depende solo del progreso y del tiempo transcurrido lo mantengo coherente al pausar y al reabrir sin guardar nada específico del dibujo.

Las pruebas encontraron un error que no se veía leyendo el código: la primera vez que se abría la extensión, getItem() devolvía null y el temporizador empezaba en 00:00. Lo resolví con la función leer(), que usa un valor por defecto cuando el dato no existe.

También tuve que precisar que storeItem() usa localStorage ya que no depende del permiso storage, sino que el permiso se mantiene, pero documentar su alcance real evita confusiones y deja preparada la mejora de las notificaciones.

10. Enlace a Github

- https://github.com/kramery/desarrollodeaplicacionesinteractivasUOC

11. Enlace a Folio

- https://jzamorajim.folio.uoc.edu/2026/11/05/desarrollo-de-aplicaciones-interactivas-r3/

12. Fuentes

- MDN – browser_specific_settings (manifest.json)
- MDN – action (manifest.json)
- MDN – API storage
- MDN – API notifications
- MDN – API alarms
- Firefox Extension Workshop – Built-in consent for data collection
- Firefox Extension Workshop – Manifest V3 migration guide
- p5.js – referencia de storeItem()
- p5.js – referencia de getItem()